

JSP include action tag

Include action tag is used for including another resource to the current JSP page. The included resource can be a static page in HTML, JSP page or Servlet. We can also pass parameters and their values to the resource which we are including. Below I have shared two examples of `<jsp:include>`, one which includes a page without passing any parameters and in second example we are passing few parameters to the page which is being included.

Syntax:

1) Include along with parameters.

```
<jsp:include page="Relative_URL_of_Page">
<jsp:param ... />
<jsp:param ... />
<jsp:param ... />
...
<jsp:param ... />
</jsp:include>
```

2) Include of another resource without sharing parameters.

```
<jsp:include page="Relative_URL_of_Page" />
```

Relative_URL_of_Page would be the page name if the page resides in the same directory where the current JSP resides.

Example 1: `<jsp:include>` without parameters

In this example we will use `<jsp:include>` action tag without parameters. As a result the page will be included in the current JSP page as it is:

index.jsp

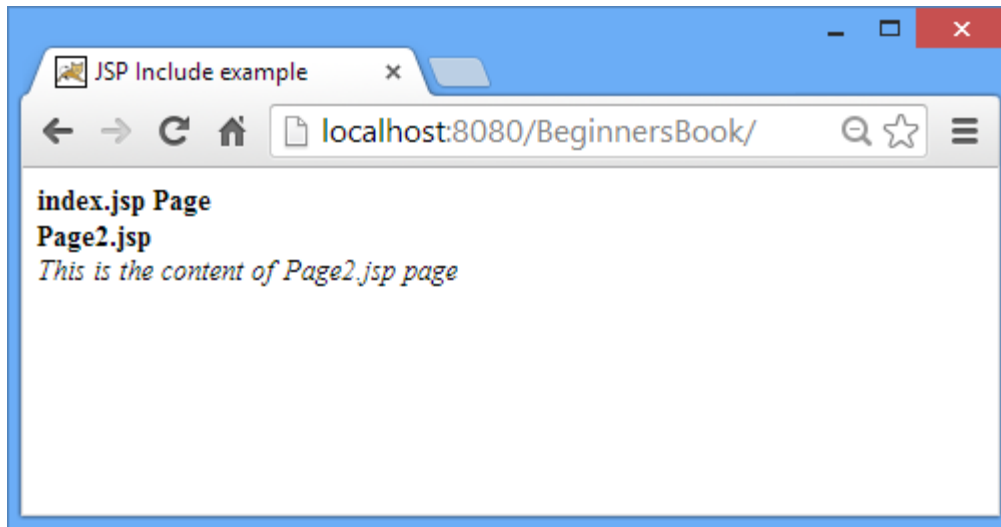
```
<html>
<head>
<title>JSP Include example</title>
</head>
<body>
<b>index.jsp Page</b><br>
<jsp:include page="Page2.jsp" />
</body>
</html>
```

Page2.jsp

```
<b>Page2.jsp</b><br>
<i> This is the content of Page2.jsp page</i>
```

Output:

Content of Page2.jsp has been appended in the index.jsp.



Example 2: Use of <jsp:include> along with <jsp:param>

index.jsp

I'm using **<jsp:include>** action here along with <jsp:param> for passing parameters to the page which we are going to include.

```
<html>
<head>
<title>JSP Include example with parameters</title>
</head>
<body>
<h2>This is index.jsp Page</h2>
<jsp:include page="display.jsp">
<jsp:param name="userid" value="Chaitanya" />
<jsp:param name="password" value="Chaitanya" />
<jsp:param name="name" value="Chaitanya Pratap Singh" />
<jsp:param name="age" value="27" />
</jsp:include>
</body>
</html>
```

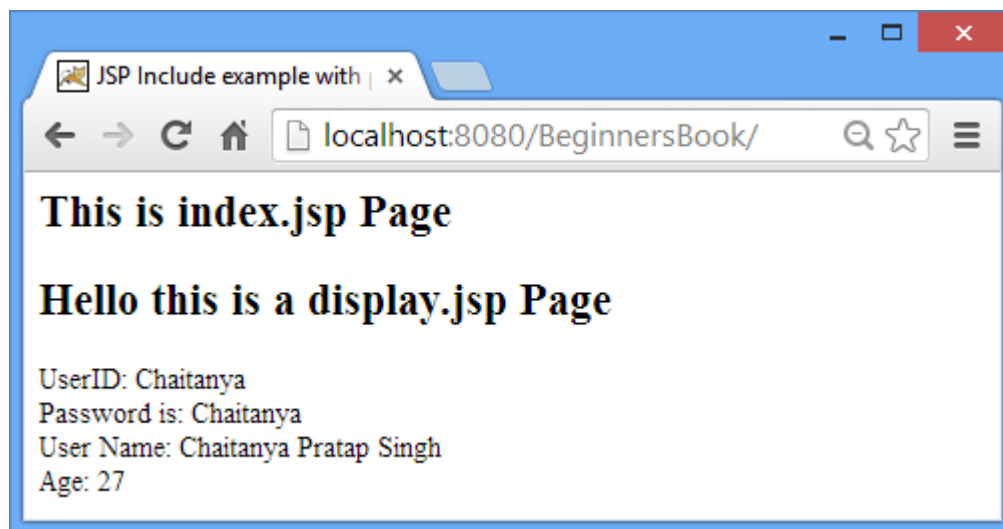
display.jsp

```
<html>
<head>
```

```
<title>Display Page</title>
</head>
<body>
<h2>Hello this is a display.jsp Page</h2>
UserID: <%=request.getParameter("userid") %><br>
Password is: <%=request.getParameter("password") %><br>
User Name: <%=request.getParameter("name") %><br>
Age: <%=request.getParameter("age") %>
</body>
</html>
```

Output:

As you can see that the content of display.jsp has been included in index.jsp. Also, the parameters we have passed are displaying correctly in the included page.



JSP forward action tag

JSP forward action tag is used for forwarding a request to the another resource (It can be a JSP, static page such as html or Servlet). Request can be forwarded with or without parameter. In this tutorial we will see examples of `<jsp:forward>` action tag.

Syntax:

- 1) Forwarding along with parameters.

```
<jsp:forward page="display.jsp">
<jsp:param ... />
<jsp:param ... />
<jsp:param ... />
...
<jsp:param ... />
</jsp:forward>
```

2) Forwarding without parameters.

```
<jsp:forward page="Relative_URL_of_Page" />
```

Relative_URL_of_Page: If page is in the same directory where the main page resides then use page name itself as I did in the below examples.

JSP Forward Example 1 - without passing parameters

In this example we are having two JSP pages – index.jsp and display.jsp. We have used <jsp:forward> action tag in index.jsp for forwarding the request to display.jsp. Here we are not passing any parameters while using the action tag. In the next example we will pass the parameters as well to another resource.

index.jsp

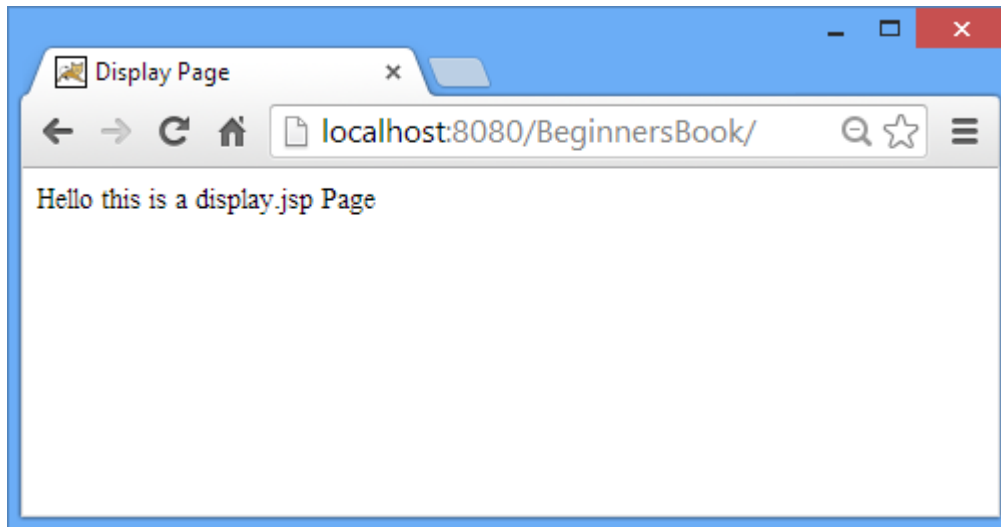
```
<html>
<head>
<title>JSP forward action tag example</title>
</head>
<body>
<p align="center">My main JSP page</p>
<jsp:forward page="display.jsp" />
</body>
</html>
```

display.jsp

```
<html>
<head>
<title>Display Page</title>
</head>
<body>
Hello this is a display.jsp Page
</body>
</html>
```

Output:

Below is the output of above code. It is basically the content of display.jsp, which clearly shows that index.jsp didn't display as it forwarded the request to the **display.jsp** page.



JSP Forward Example 2 - with parameters

Here we are passing the parameters along with forward request. For passing parameters we are using `<jsp:param>` action tag. In this example we are passing 4 parameters along with forward and later we are displaying them on the forwarded page. In order to fetch the parameters on display.jsp page we are using `getParameter` method of [request implicit object](#).

index.jsp

```
<html>
<head>
<title>JSP forward example with parameters</title>
</head>
<body>
<jsp:forward page="display.jsp">
<jsp:param name="name" value="Chaitanya" />
<jsp:param name="site" value="BeginnersBook.com" />
<jsp:param name="tutorialname" value="jsp forward action" />
<jsp:param name="reqcamefrom" value="index.jsp" />
</jsp:forward>
</body>
</html>
```

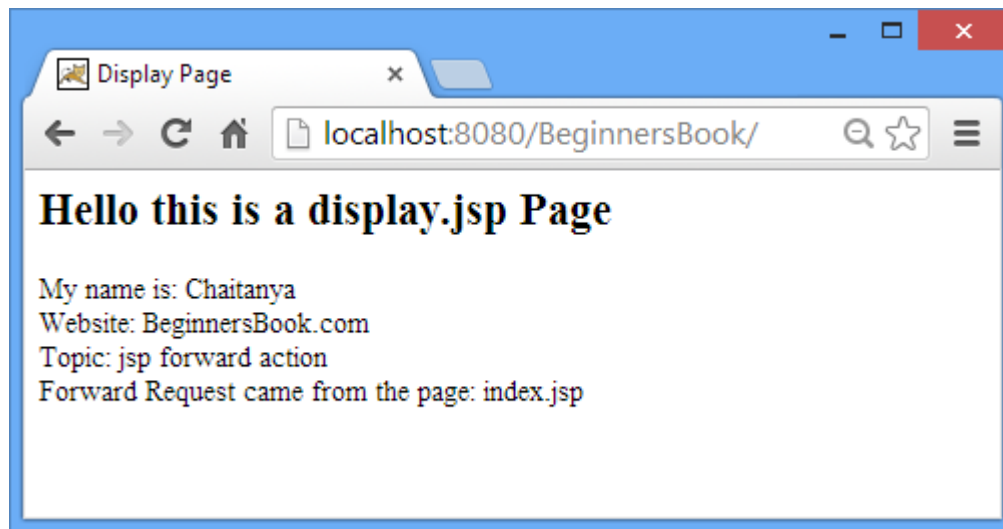
display.jsp

```
<html>
<head>
<title>Display Page</title>
</head>
<body>
<h2>Hello this is a display.jsp Page</h2>
My name is: <%=request.getParameter("name") %><br>
```

```
Website: <%=request.getParameter("site") %><br>
Topic: <%=request.getParameter("tutorialname") %><br>
Forward Request came from the page: <%=request.getParameter("reqcamefrom") %>
</body>
</html>
```

Output:

Above code directly displayed display.jsp page, which is displaying the parameters passed from index.jsp page.



Session Implicit Object

Session is most frequently used implicit object in JSP. The main usage of it to gain access to all the user's data till the user session is active.

Methods of session Implicit Object

1. **setAttribute(String, object)** – This method is used to save an object in session by assigning a unique string to the object. Later, the object can be accessed from the session by using the same String till the session is active. setAttribute and getAttribute are the two most frequently used methods while dealing with session in JSP.
2. **getAttribute(String name)** – The object stored by setAttribute method is fetched from session using getAttribute method. For example if we need to access userid on every jsp page till the session is active then we should store the user-id in session using setAttribute method and can be accessed using getAttribute method whenever needed.

3. **removeAttribute(String name)** – The objects which are stored in session can be removed from session using this method. Pass the unique string identifier as removeAttribute's method.
4. **getAttributeNames** – It returns all the objects stored in session. Basically, it results in an enumeration of objects.
5. **getCreationTime** – This method returns the session creation time, the time when session got initiated (became active).
6. **getId** – Servlet container assigns a unique string identifier to session while creation of it. **getId** method returns that unique string identifier.
7. **isNew()** – Used to check whether the session is new. It returns Boolean value (true or false). Mostly used to track whether the cookies are enabled on client side. If cookies are disabled the session.isNew() method would always return **true**.
8. **invalidate()** – It kills a session and breaks the association of session with all the stored objects.
9. **getMaxInactiveInterval** – Returns session's maximum inactivate time interval in seconds.
10. **getLastAccessedTime** – Generally used to know the last accessed time of a session.

Session Implicit Object Example

The below html page would display a text box along with a submit button. The submit action would transfer the control to session.jsp page.

[index.html](#)

```
<html>
<head>
<title>Welcome Page: Enter your name</title>
</head>
<body>
<form action="session.jsp">
<input type="text" name="inputname">
<input type="submit" value="click here!!"><br/>
</form>
</body>
</html>
```

The **session.jsp** page displays the name which user has entered in the index page and it stores the the same variable in the **session object** so that it can be fetched on any page until the session becomes inactive.

[session.jsp](#)

```

<html>
<head>
<title>Passing the input value to a session variable</title>
</head>
<body>
<%
String uname=request.getParameter("inputname");
out.print("Welcome " + uname);
session.setAttribute("sessname",uname);
%>
<a href="output.jsp">Check Output Page Here </a>
</body>
</html>

```

In this page we are fetching the variable's value from session object and displaying it.

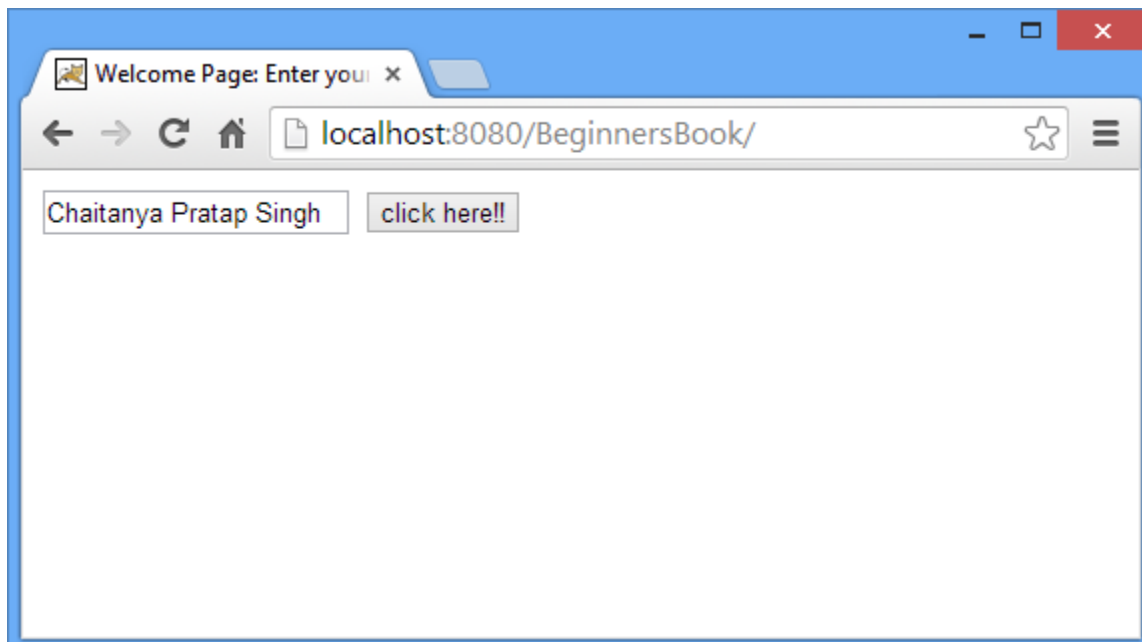
output.jsp

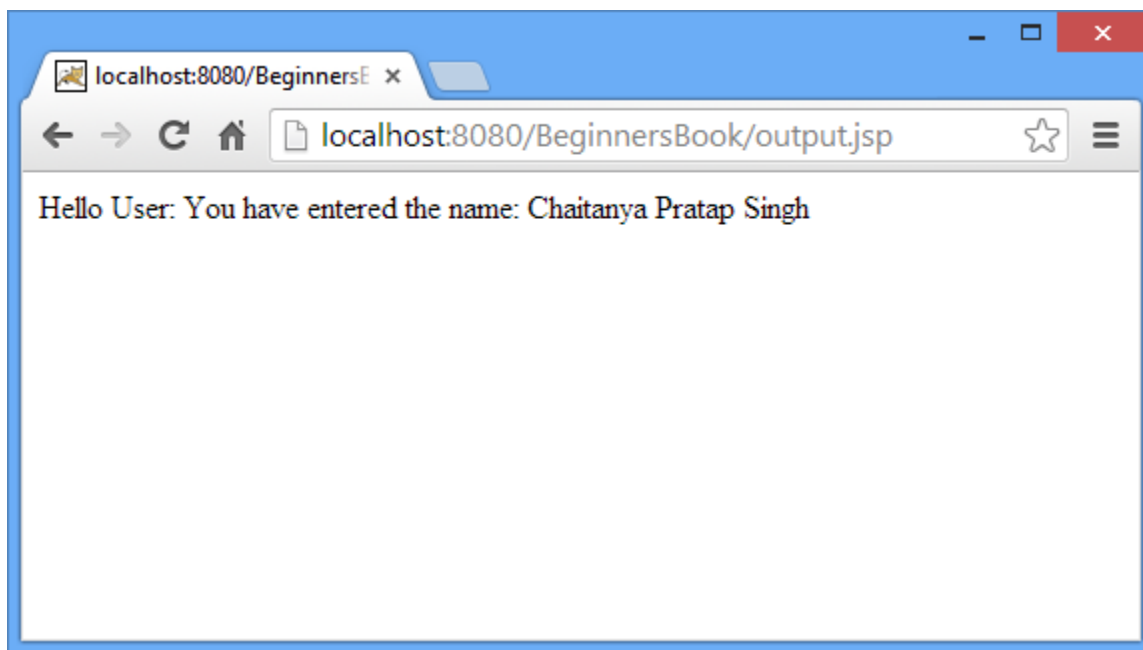
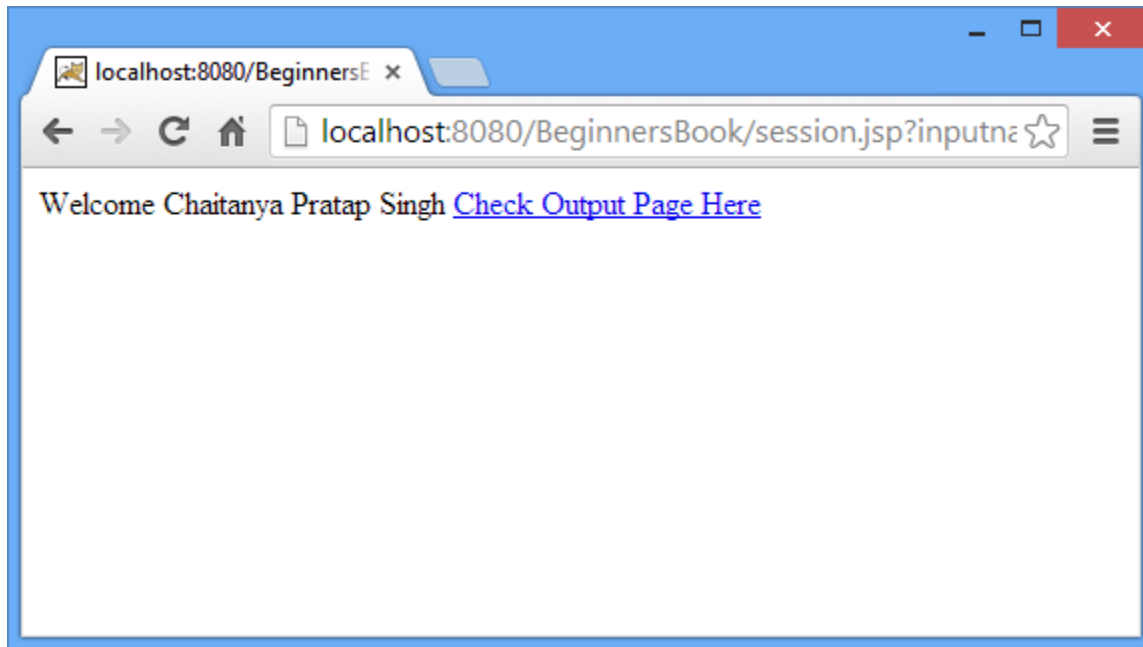
```

<html>
<head>
<title>Output page: Fetching the value from session</title>
</head>
<body>
<%
String name=(String)session.getAttribute("sessname");
out.print("Hello User: You have entered the name: "+name);
%>
</body>
</html>

```

Output Screens





Exception Implicit Object

In this tutorial, we will discuss exception implicit object of JSP. It's an instance of `java.lang.Throwable` and frequently used for exception handling in JSP. This object is only available for error pages, which means a JSP page should have `isErrorPage` to `true` in order to use exception implicit object. Let's understand this with the help of below example –

Exception implicit Object Example

In this example we are taking two integer inputs from user and then we are performing division between them. We have used exception implicit object to handle any kind of exception in the below example.

index.html

```
<html>
<head>
<title>Enter two Integers for Division</title>
</head>
<body>
<form action="division.jsp">
Input First Integer:<input type="text" name="firstnum" />
Input Second Integer:<input type="text" name="secondnum" />
<input type="submit" value="Get Results"/>
</form>
</body>
</html>
```

Here we have specified exception.jsp as errorPage which means if any exception occurs in this JSP page, the control will immediately transferred to the exception.jsp JSP page. Note: We have used **errorPage attribute of Page Directive** to specify the exception handling JSP page (<%@ page errorPage="exception.jsp" %>).

division.jsp

```
<%@ page errorPage="exception.jsp" %>
<%
String num1=request.getParameter("firstnum");
String num2=request.getParameter("secondnum");
int v1= Integer.parseInt(num1);
int v2= Integer.parseInt(num2);
int res= v1/v2;
out.print("Output is: "+ res);
%>
```

In the below JSP page we have set **isErrorPage to true** which is also an attribute of Page directive, used for making a page eligible for exception handling. Since this page is defined as a exception page in division.jsp, in case of any exception condition this page will be invoked. Here we are displaying the error message to the user using **exception implicit object**.

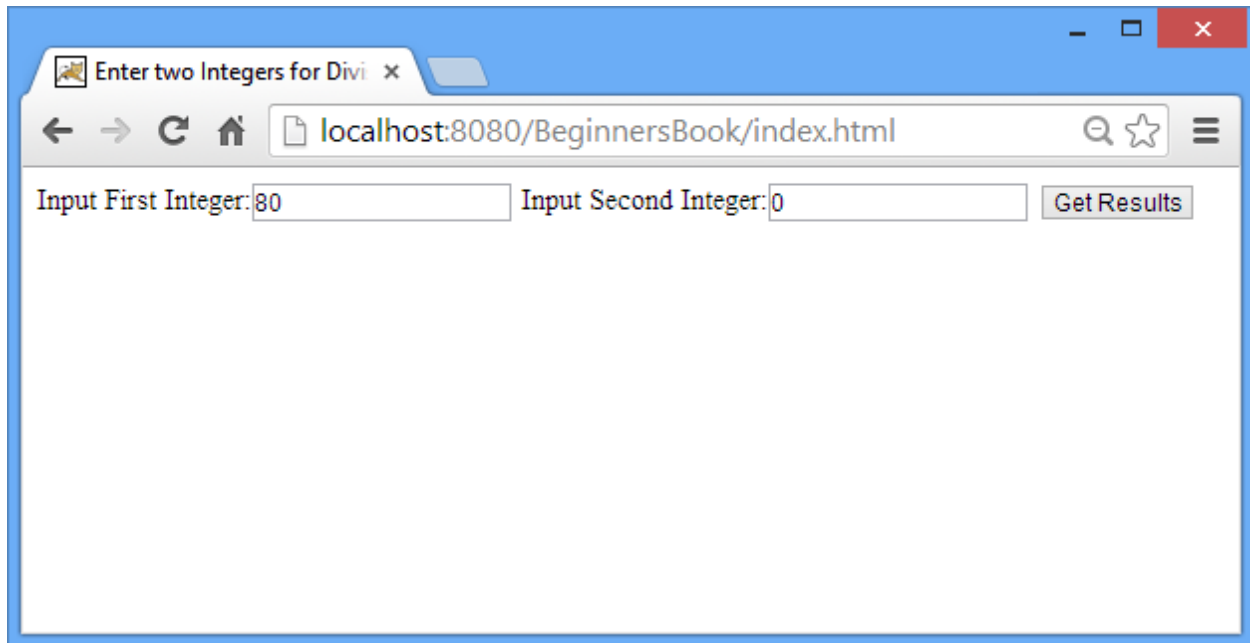
exception.jsp

```
<%@ page isErrorPage="true" %>
```

Got this Exception: <%= exception %>
Please correct the input data.

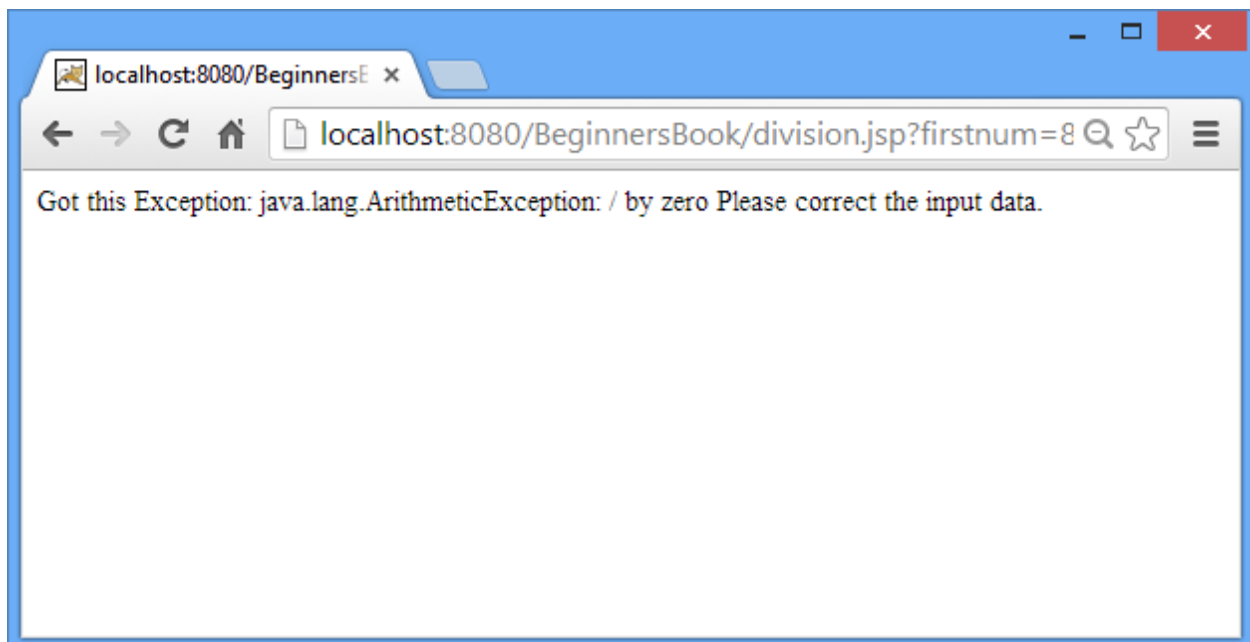
Output

Screen with two input fields for two integer numbers.



A screenshot of a web browser window. The title bar says "Enter two Integers for Divi x". The address bar shows "localhost:8080/BeginnersBook/index.html". The page content has two input fields: "Input First Integer:" followed by a text box containing "80", and "Input Second Integer:" followed by a text box containing "0". To the right of these fields is a button labeled "Get Results".

Arithmetic Exception message when we have provided the second integer as zero.



A screenshot of a web browser window. The title bar says "localhost:8080/BeginnersE x". The address bar shows "localhost:8080/BeginnersBook/division.jsp?firstnum=8". The page content displays the message: "Got this Exception: java.lang.ArithmeticException: / by zero Please correct the input data."

JSP - Database Access

In this chapter, we will discuss how to access database with JSP. We assume you have good understanding on how JDBC application works. Before starting with database access through a JSP, make sure you have proper JDBC environment setup along with a database.

For more detail on how to access database using JDBC and its environment setup you can go through our [JDBC Tutorial](#).

To start with basic concept, let us create a table and create a few records in that table as follows –

Create Table

To create the **Employees** table in the EMP database, use the following steps –

Step 1

Open a **Command Prompt** and change to the installation directory as follows –

```
C:\>  
C:\>cd Program Files\MySQL\bin  
C:\Program Files\MySQL\bin>
```

Step 2

Login to the database as follows –

```
C:\Program Files\MySQL\bin>mysql -u root -p  
Enter password: *****  
mysql>
```

Step 3

Create the **Employee** table in the **TEST** database as follows – –

```
mysql> use TEST;
```

```
mysql> create table Employees
(
    id int not null,
    age int not null,
    first varchar (255),
    last varchar (255)
);
Query OK, 0 rows affected (0.08 sec)
mysql>
```

Create Data Records

Let us now create a few records in the **Employee** table as follows –

```
mysql> INSERT INTO Employees VALUES (100, 18, 'Zara', 'Ali');
Query OK, 1 row affected (0.05 sec)

mysql> INSERT INTO Employees VALUES (101, 25, 'Mahnaz', 'Fatma');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Employees VALUES (102, 30, 'Zaid', 'Khan');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Employees VALUES (103, 28, 'Sumit', 'Mittal');
Query OK, 1 row affected (0.00 sec)

mysql>
```

SELECT Operation

Following example shows how we can execute the **SQL SELECT** statement using JTSL in JSP programming –

```
<%@ page import = "java.io.*,java.util.*,java.sql.*"%>
<%@ page import = "javax.servlet.http.*,javax.servlet.*" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix = "c"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix = "sql"%>

<html>

  <head>

    <title>SELECT Operation</title>

  </head>

  <body>

    <sql:setDataSource var = "snapshot" driver = "com.mysql.jdbc.Driver"
      url = "jdbc:mysql://localhost/TEST"
      user = "root" password = "pass123"/>

    <sql:query dataSource = "${snapshot}" var = "result">
      SELECT * from Employees;
    </sql:query>

    <table border = "1" width = "100%">
      <tr>
        <th>Emp ID</th>
        <th>First Name</th>
        <th>Last Name</th>
        <th>Age</th>
      </tr>

      <c:forEach var = "row" items = "${result.rows}">
```

```

        <tr>

            <td><c:out value = "${row.id}"/></td>

            <td><c:out value = "${row.first}"/></td>

            <td><c:out value = "${row.last}"/></td>

            <td><c:out value = "${row.age}"/></td>

        </tr>

    </c:forEach>

</table>

</body>
</html>

```

Access the above JSP, the following result will be displayed –

Emp ID	First Name	Last Name	Age
100	Zara	Ali	18
101	Mahnaz	Fatma	25
102	Zaid	Khan	30
103	Sumit	Mittal	28

INSERT Operation

Following example shows how we can execute the SQL INSERT statement using JTSL in JSP programming –

```

<%@ page import = "java.io.*,java.util.*,java.sql.*"%>

<%@ page import = "javax.servlet.http.*,javax.servlet.*" %>

```

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix = "c"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix = "sql"%>

<html>

    <head>

        <title>JINSERT Operation</title>

    </head>

    <body>

        <sql:setDataSource var = "snapshot" driver = "com.mysql.jdbc.Driver"
            url = "jdbc:mysql://localhost/TEST"
            user = "root" password = "pass123"/>

        <sql:update dataSource = "${snapshot}" var = "result">
            INSERT INTO Employees VALUES (104, 2, 'Nuha', 'Ali');
        </sql:update>

        <sql:query dataSource = "${snapshot}" var = "result">
            SELECT * from Employees;
        </sql:query>

        <table border = "1" width = "100%">

            <tr>

                <th>Emp ID</th>

                <th>First Name</th>

                <th>Last Name</th>

                <th>Age</th>

            </tr>
```



```

<c:forEach var = "row" items = "${result.rows}">
    <tr>
        <td><c:out value = "${row.id}"/></td>
        <td><c:out value = "${row.first}"/></td>
        <td><c:out value = "${row.last}"/></td>
        <td><c:out value = "${row.age}"/></td>
    </tr>
</c:forEach>
</table>

</body>
</html>

```

Access the above JSP, the following result will be displayed –

Emp ID	First Name	Last Name	Age
100	Zara	Ali	18
101	Mahnaz	Fatma	25
102	Zaid	Khan	30
103	Sumit	Mittal	28
104	Nuha	Ali	2

DELETE Operation

Following example shows how we can execute the **SQL DELETE** statement using JTSL in JSP programming –

```
<%@ page import = "java.io.*,java.util.*,java.sql.*"%>
<%@ page import = "javax.servlet.http.*,javax.servlet.*" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix = "c"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix = "sql"%>

<html>

    <head>

        <title>DELETE Operation</title>

    </head>

    <body>

        <sql:setDataSource var = "snapshot" driver = "com.mysql.jdbc.Driver"
            url = "jdbc:mysql://localhost/TEST"
            user = "root" password = "pass123"/>

        <c:set var = "empId" value = "103"/>

        <sql:update dataSource = "${snapshot}" var = "count">
            DELETE FROM Employees WHERE Id = ?
            <sql:param value = "${empId}" />
        </sql:update>

        <sql:query dataSource = "${snapshot}" var = "result">
            SELECT * from Employees;
        </sql:query>

        <table border = "1" width = "100%">

            <tr>
```

```

        <th>Emp ID</th>

        <th>First Name</th>

        <th>Last Name</th>

        <th>Age</th>

    </tr>

    <c:forEach var = "row" items = "${result.rows}">
        <tr>
            <td><c:out value = "${row.id}"/></td>
            <td><c:out value = "${row.first}"/></td>
            <td><c:out value = "${row.last}"/></td>
            <td><c:out value = "${row.age}"/></td>
        </tr>
    </c:forEach>
</table>

</body>
</html>

```

Access the above JSP, the following result will be displayed –

Emp ID	First Name	Last Name	Age
100	Zara	Ali	18
101	Mahnaz	Fatma	25
102	Zaid	Khan	30

UPDATE Operation

Following example shows how we can execute the **SQL UPDATE** statement using JTSL in JSP programming –

```
<%@ page import = "java.io.*,java.util.*,java.sql.*"%>
<%@ page import = "javax.servlet.http.*,javax.servlet.*" %>
<%@ taglib uri = "http://java.sun.com/jsp/jstl/core" prefix = "c"%>
<%@ taglib uri = "http://java.sun.com/jsp/jstl/sql" prefix = "sql"%>

<html>

  <head>

    <title>DELETE Operation</title>

  </head>

  <body>

    <sql:setDataSource var = "snapshot" driver = "com.mysql.jdbc.Driver"
      url = "jdbc:mysql://localhost/TEST"
      user = "root" password = "pass123"/>

    <c:set var = "empId" value = "102"/>

    <sql:update dataSource = "${snapshot}" var = "count">
      UPDATE Employees SET WHERE last = 'Ali'
      <sql:param value = "${empId}" />
    </sql:update>

    <sql:query dataSource = "${snapshot}" var = "result">
      SELECT * from Employees;
```

```

</sql:query>

<table border = "1" width = "100%">

    <tr>

        <th>Emp ID</th>

        <th>First Name</th>

        <th>Last Name</th>

        <th>Age</th>

    </tr>

    <c:forEach var = "row" items = "${result.rows}">

        <tr>

            <td><c:out value = "${row.id}"/></td>

            <td><c:out value = "${row.first}"/></td>

            <td><c:out value = "${row.last}"/></td>

            <td><c:out value = "${row.age}"/></td>

        </tr>

    </c:forEach>

</table>

</body>
</html>

```

Access the above JSP, the following result will be displayed –

Emp ID	First Name	Last Name	Age
100	Zara	Ali	18

101	Mahnaz	Fatma	25
102	Zaid	Ali	30